EXPERIMENT - 69

1. Problem Statement

Write a recursive function to generate the **first n Fibonacci numbers**. Also count the number of recursive calls made.

2. Algorithm

Main Program

Step 1: START
Step 2: Set callCount = 0 and read n.
Step 3: Repeat for i = 0 to n − 1: call fib(i) and display the result.
Step 4: Display callCount.
Step 5: STOP

Function fib(k)

Step 1: Increase callCount by 1.
Step 2: If k <= 1, return k.
Step 3: Return fib(k − 1) + fib(k − 2).

3. Test Case Table

#	n	Expected Output	Actual Output	Result
1	1	Fibonacci series: 0 Number of recursive calls: 1	Fibonacci series: 0 Number of recursive calls: 1	**PASS**
2	2	Fibonacci series: 0 1 Number of recursive calls: 2	Fibonacci series: 0 1 Number of recursive calls: 2	**PASS**
3	5	Fibonacci series: 0 1 1 2 3 Number of recursive calls: 19	Fibonacci series: 0 1 1 2 3 Number of recursive calls: 19	**PASS**
4	7	Fibonacci series: 0 1 1 2 3 5 8 Number of recursive calls: 59	Fibonacci series: 0 1 1 2 3 5 8 Number of recursive calls: 59	**PASS**
5	10	Fibonacci series: 0 1 1 2 3 5 8 13 21 34 Number of recursive calls: 276	Fibonacci series: 0 1 1 2 3 5 8 13 21 34 Number of recursive calls: 276	**PASS**

4. Source Code

fib69.c

```
/* Experiment 69: first n Fibonacci numbers using recursion,
   and a count of the calls made. */
#include <stdio.h>

static int callCount = 0;    /* increased on every call of fib() */

int fib(int k);

int main(void)
{
    int n, i;

    printf("Enter the value of n: ");
    scanf("%d", &n);

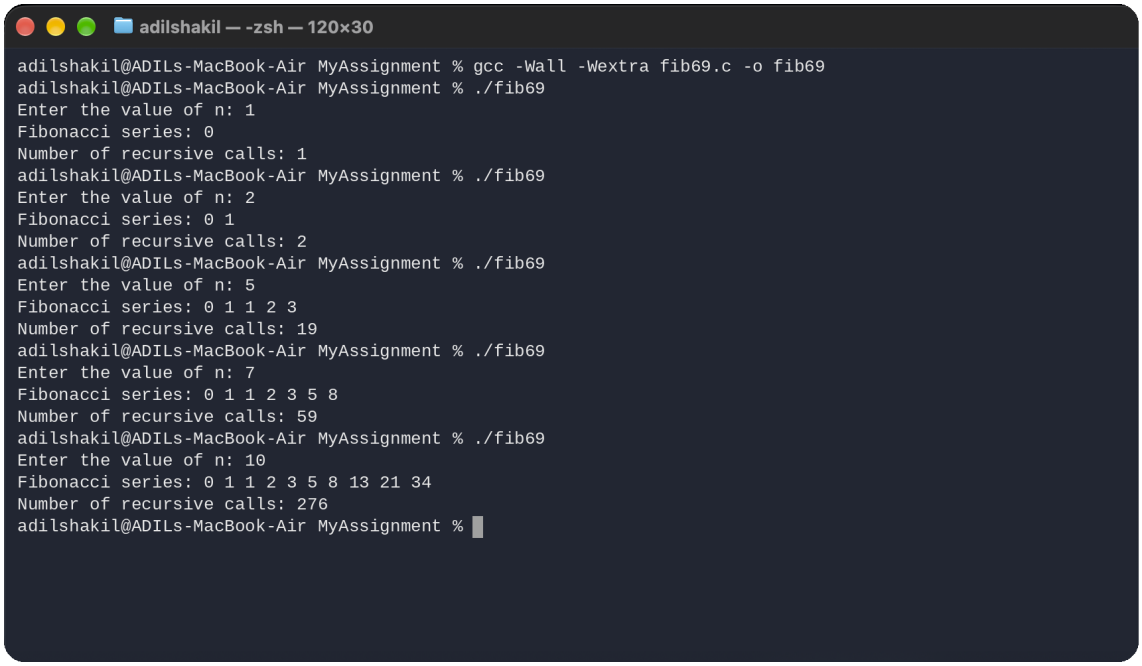
    printf("Fibonacci series:");
    for (i = 0; i < n; i++)
        printf(" %d", fib(i));
    printf("\n");

    printf("Number of recursive calls: %d\n", callCount);
    return 0;
}

/* fib(k) = fib(k - 1) + fib(k - 2), with fib(0) = 0 and fib(1) = 1. */
int fib(int k)
{
    callCount++;
    if (k <= 1)
        return k;
    return fib(k - 1) + fib(k - 2);
}
```

5. Compilation and Output

Compiled with `gcc -Wall -Wextra fib69.c -o fib69` so the small warnings would show up too. It built clean.



All five passed.